

# PROJECT: Détection de fraude

Préparer par :  
DOUAE BENABOU

Encadrée par :  
Nadia Idrissizougari



# Plan

Introduction(avec  
la problématique  
et l'objectif )

Modele 1:knn

Modele 2 : svm  
(3 noyaux)

Arbre de décision

Analyse des risques  
éthiques

Conclusion

# Introduction

Dans un contexte de numérisation croissante des services financiers, la lutte contre la fraude est devenue un enjeu de sécurité critique. Ce projet explore la puissance du **Machine Learning** pour automatiser l'identification des comportements suspects à travers une étude comparative de plusieurs architectures algorithmiques.



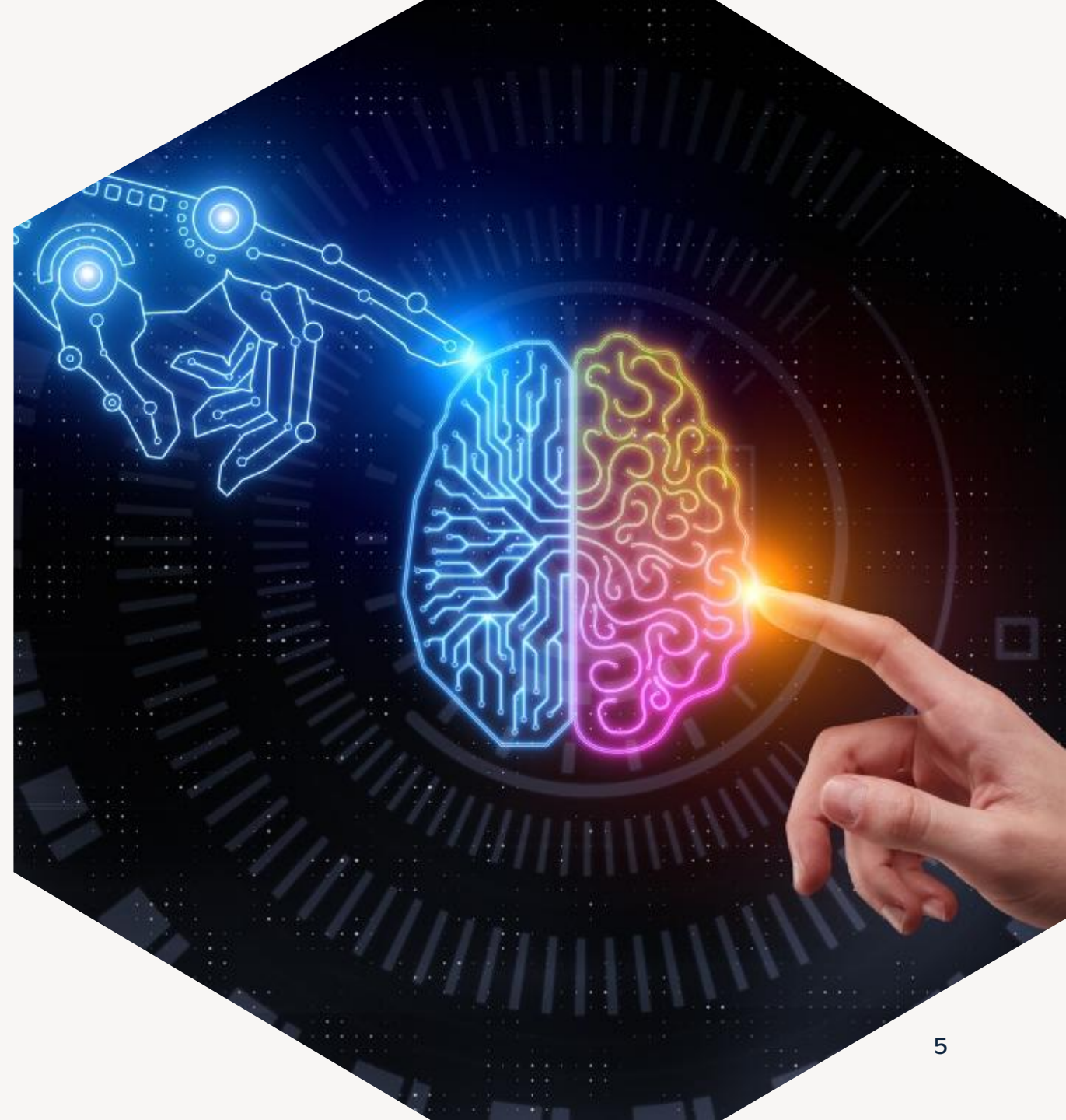
# Problematique

Comment le Machine Learning peut-il transformer la détection de fraude d'une simple réaction à une protection prédictive et fiable ?

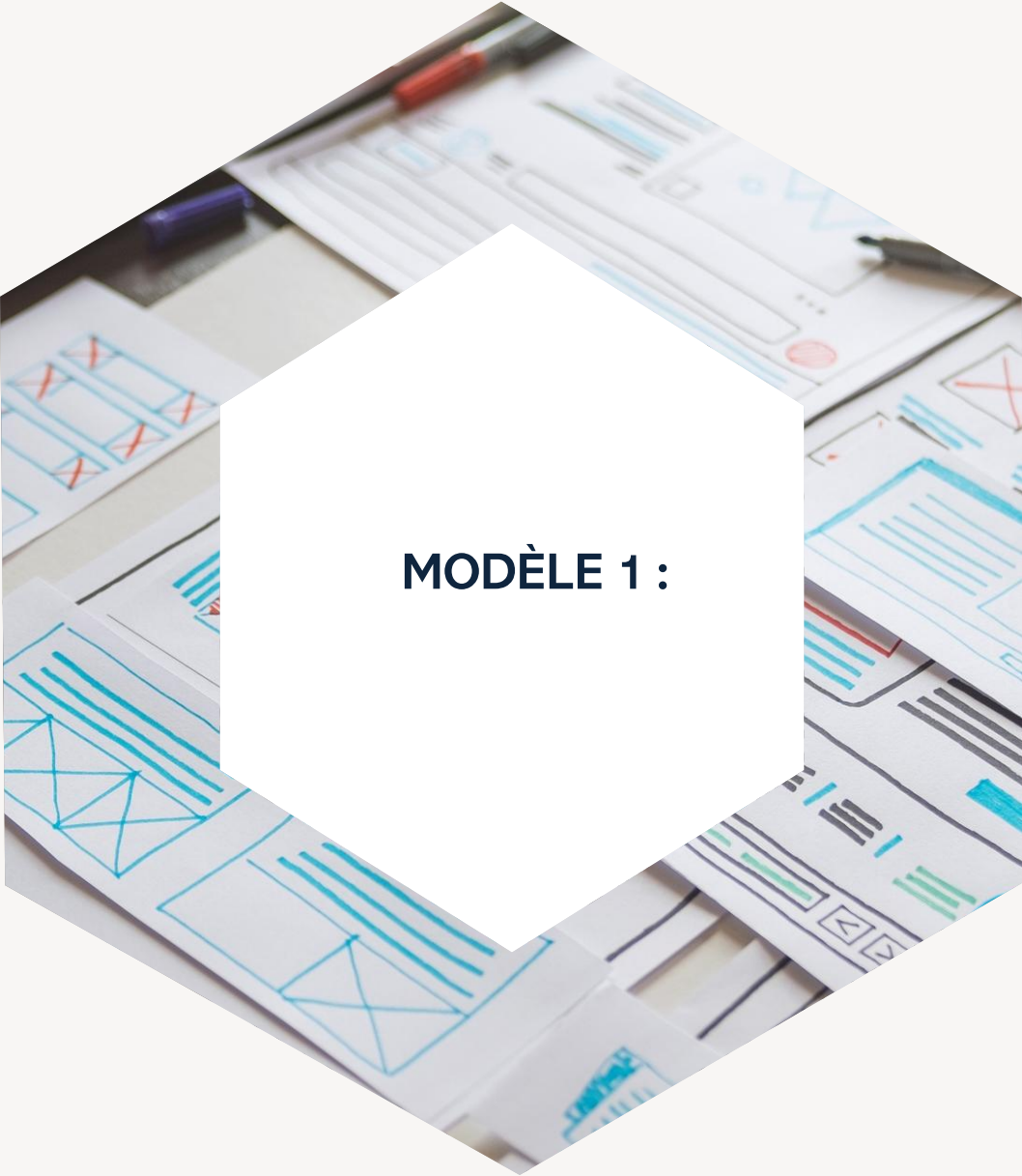


# Objectifs

1. **Analyse Comparative des Algorithmes:** tester (KNN, SVM, Arbre de Décision) pour identifier celui qui capture le mieux les schémas de fraude.
2. **Optimisation Mathématique (GridSearchCV):** Automatiser la recherche des meilleurs réglages (hyperparamètres) pour chaque modèle.
3. **Fiabilité et Robustesse:** Éliminer tout risque de **surapprentissage (overfitting)** grâce à l'élagage des arbres et à l'analyse des courbes d'apprentissage.
4. **Aide à la Décision (Déploiement):** Proposer un outil "éthique" où l'IA ne remplace pas l'humain mais lui fournit une recommandation claire (Fraude / Légitime).







MODÈLE 1 :

# **KNN (K-Nearest Neighbors)**

**Principe :** Le KNN classifie une transaction en observant les « K » transactions les plus similaires (voisins) dans l'espace des données.

## 1. Configuration et Optimisation

- ✓ Utilisation du **StandardScaler** pour normaliser les données

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

- ✓ Recherche d'hyperparamètres (GridSearchCV)

```
param_grid = {'n_neighbors': np.arange(1, 21)}
grid_knn = GridSearchCV(KNeighborsClassifier(), param_grid, cv=5, scoring='f1')
grid_knn.fit(X_train_scaled, y_train)
best_k = grid_knn.best_params_['n_neighbors']
print(f"Meilleur paramètre K trouvé : {best_k}")
```

```
Meilleur paramètre K trouvé : 5
```

## 2. Performances obtenues (Données de Test)

- **Précision (Accuracy)**
- **Recall (Détection de fraude)**
- **Analyse de la Matrice de Confusion :**
- **Vrais Positifs** : 10 fraudes correctement identifiées.
- **Faux Négatifs** : 11 fraudes manquées (un point d'amélioration identifié).
- **Faux Positifs** : Seulement 2 clients bloqués par erreur.

```
final_knn = KNeighborsClassifier(n_neighbors=best_k)
final_knn.fit(X_train_scaled, y_train)
y_pred = final_knn.predict(X_test_scaled)
# Calcul des métriques
print("--- Résultats du Modèle KNN ---")
print(f"Accuracy : {accuracy_score(y_test, y_pred):.4f}")
print(f"Precision : {precision_score(y_test, y_pred):.4f}")
print(f"Recall : {recall_score(y_test, y_pred):.4f}")
print("\nMatrice de Confusion :")
print(confusion_matrix(y_test, y_pred))
```

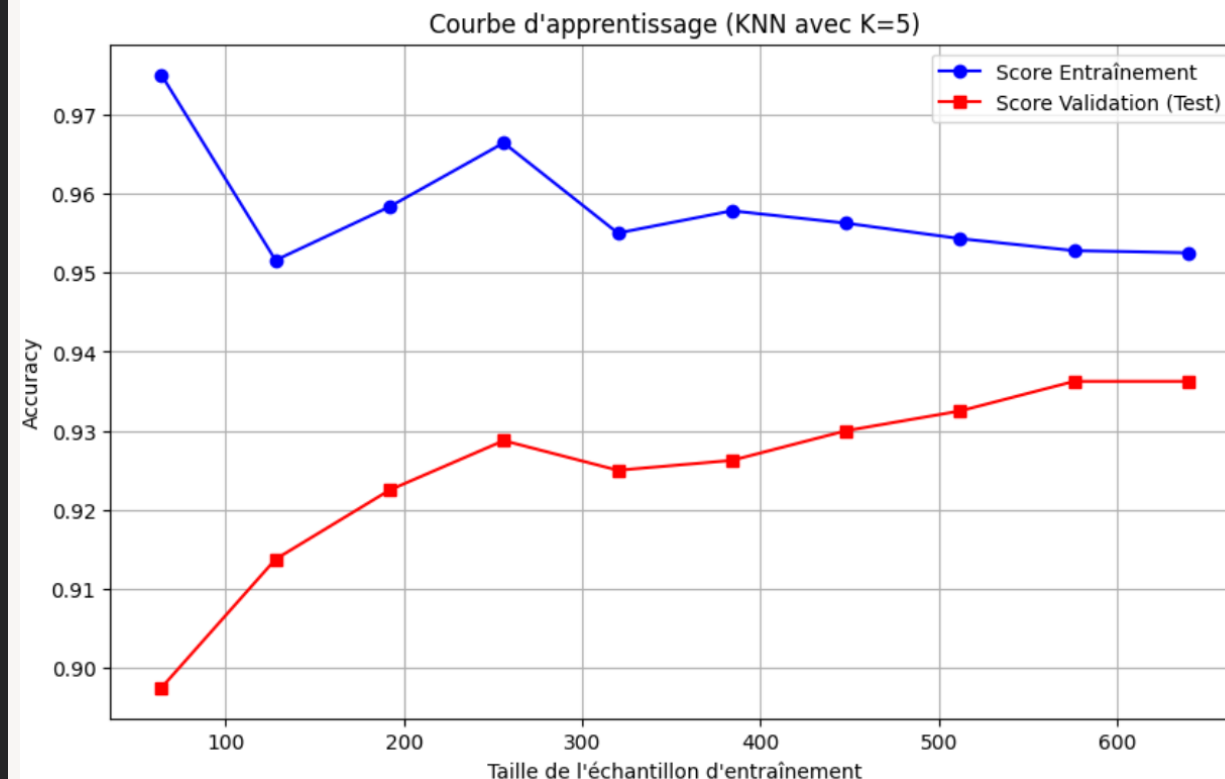
```
--- Résultats du Modèle KNN ---
Accuracy : 0.9350
Precision : 0.8333
Recall : 0.4762

Matrice de Confusion :
[[177  2]
 [ 11 10]]
```



### 3. Courbe d'apprentissage

```
def plot_learning_curve(model, X, y):  
    train_sizes, train_scores, test_scores = learning_curve(  
        model, X, y, cv=5, scoring='accuracy', n_jobs=-1,  
        train_sizes=np.linspace(0.1, 1.0, 10)  
    )  
    # Calcul des moyennes et écart-types  
    train_mean = np.mean(train_scores, axis=1)  
    test_mean = np.mean(test_scores, axis=1)  
    plt.figure(figsize=(10, 6))  
    plt.plot(train_sizes, train_mean, label='Score Entraînement', color='blue', marker='o')  
    plt.plot(train_sizes, test_mean, label='Score Validation (Test)', color='red', marker='s')  
    plt.title(f'Courbe d\'apprentissage (KNN avec K={best_k})')  
    plt.xlabel('Taille de l\'échantillon d\'entraînement')  
    plt.ylabel('Accuracy')  
    plt.legend(loc='best')  
    plt.grid(True)  
    plt.show()  
plot_learning_curve(final_knn, X_train_scaled, y_train)
```



- **Stabilité** : Le modèle est robuste.
- **Fiabilité** : Pas d'apprentissage par cœur.
- **Validation** : Le choix de K=5 est graphiquement justifié.

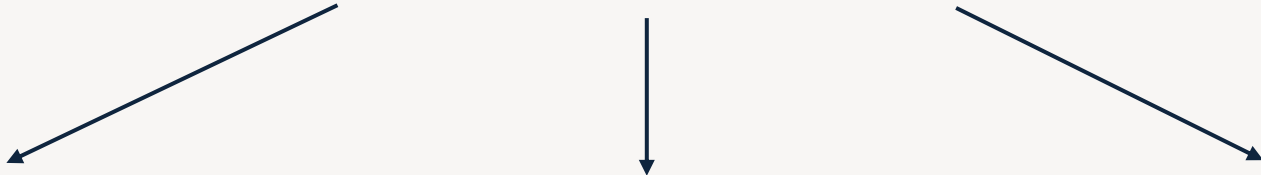


MODÈLE 2:

# **SVM** **(Support Vector** **Machine)**

**Principe :** Le SVM cherche à tracer une "frontière" (hyperplan) qui sépare les transactions frauduleuses des normales avec la marge de sécurité la plus grande possible.

### **3 Noyaux (Kernels)**



```
graph TD; A[3 Noyaux (Kernels)] --> B[Linéaire :  
Pour les séparations  
simples par une ligne  
droite.]; A --> C[Polynomial :  
Pour des relations courbes  
plus complexes.]; A --> D[RBF (Radial Basis Function) :  
Le noyau le plus puissant qui  
crée des frontières flexibles  
autour des zones de fraude.];
```

#### **Linéaire :**

Pour les séparations simples par une ligne droite.

#### **Polynomial :**

Pour des relations courbes plus complexes.

#### **RBF (Radial Basis Function) :**

Le noyau le plus puissant qui crée des frontières flexibles autour des zones de fraude.

# SVM - Le Noyau Linéaire

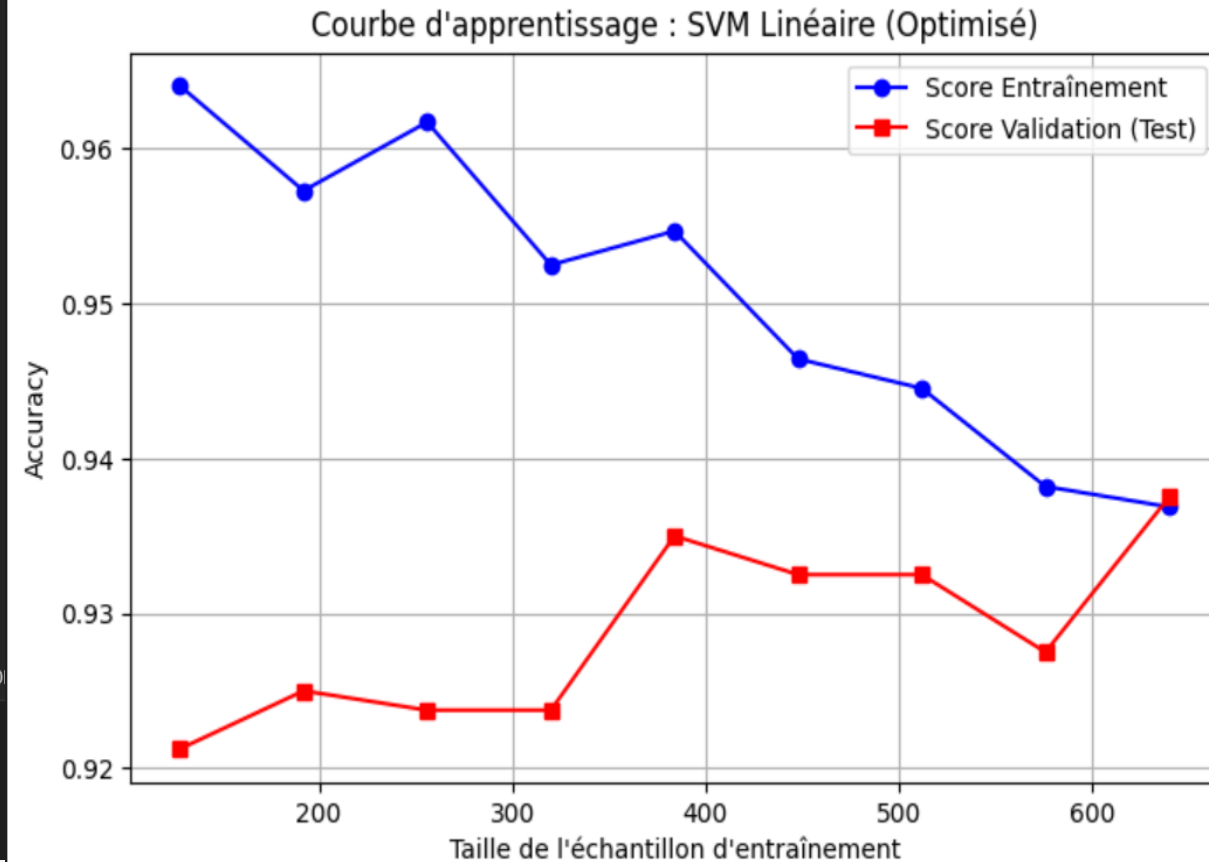
```
from sklearn.model_selection import GridSearchCV
from sklearn.svm import SVC
param_grid_lin = {'C': [0.1, 1, 10, 100]}
grid_lin = GridSearchCV(SVC(kernel='linear'), param_grid_lin, cv=5, scoring='accuracy', n_jobs=-1)
grid_lin.fit(X_train_scaled, y_train)
best_svm_lin = grid_lin.best_estimator_
acc_train_lin = best_svm_lin.score(X_train_scaled, y_train)
acc_test_lin = best_svm_lin.score(X_test_scaled, y_test)
print(f"--- SVM LINÉAIRE ---")
print(f"Meilleur paramètre : {grid_lin.best_params_}")
print(f"Accuracy Train : {acc_train_lin:.4f} | Accuracy Test : {acc_test_lin:.4f}")
import warnings
warnings.filterwarnings('ignore')
plot_individual_learning_curve(best_svm_lin, X_train_scaled, y_train, "SVM Linéaire (Optimisé)")
```

--- SVM LINÉAIRE ---

Meilleur paramètre : {'C': 0.1}

Accuracy Train : 0.9400 | Accuracy Test : 0.9250

Python



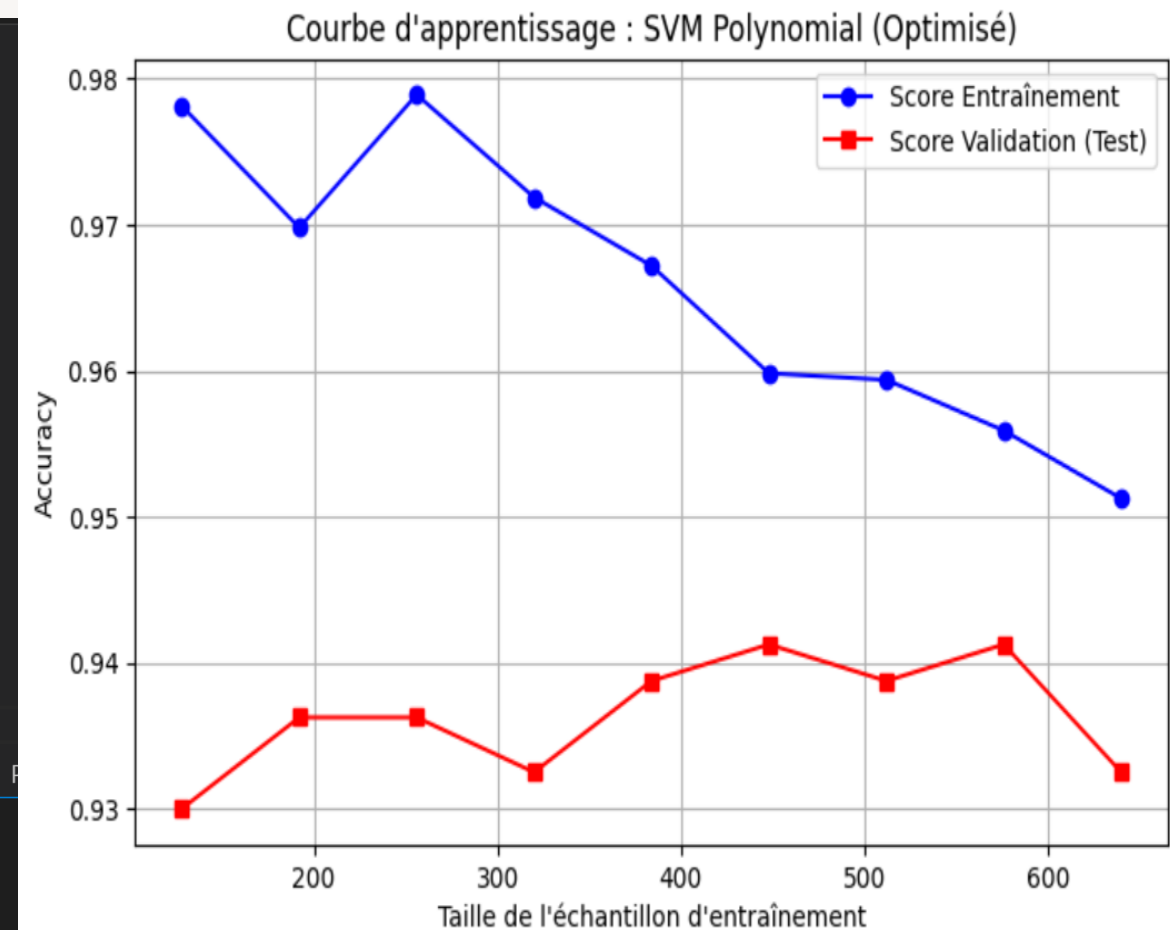
# SVM - Le Noyau Polynomial

```
# Définition des paramètres
param_grid_poly = {
    'C': [0.1, 1, 10],
    'degree': [2, 3, 4]
}

# gridsearch
grid_poly = GridSearchCV(SVC(kernel='poly'), param_grid_poly, cv=5, scoring='accuracy', n_jobs=-1)
grid_poly.fit(X_train_scaled, y_train)

# 3. Résultats
best_svm_poly = grid_poly.best_estimator_
acc_train_poly = best_svm_poly.score(X_train_scaled, y_train)
acc_test_poly = best_svm_poly.score(X_test_scaled, y_test)
print(f"--- SVM POLYNOMIAL ---")
print(f"Meilleurs paramètres : {grid_poly.best_params_}")
print(f"Accuracy Train : {acc_train_poly:.4f} | Accuracy Test : {acc_test_poly:.4f}")
import warnings
warnings.filterwarnings('ignore')
plot_individual_learning_curve(best_svm_poly, X_train_scaled, y_train, "SVM Polynomial (Optimisé)")
```

```
--- SVM POLYNOMIAL ---
Meilleurs paramètres : {'C': 1, 'degree': 3}
Accuracy Train : 0.9537 | Accuracy Test : 0.9350
```





## SVM - Le Noyau RBF (Le plus performant)

```
# 1. Définition des paramètres
param_grid_rbf = {
    'C': [0.1, 1, 10, 100],
    'gamma': [1, 0.1, 0.01, 0.001]
}

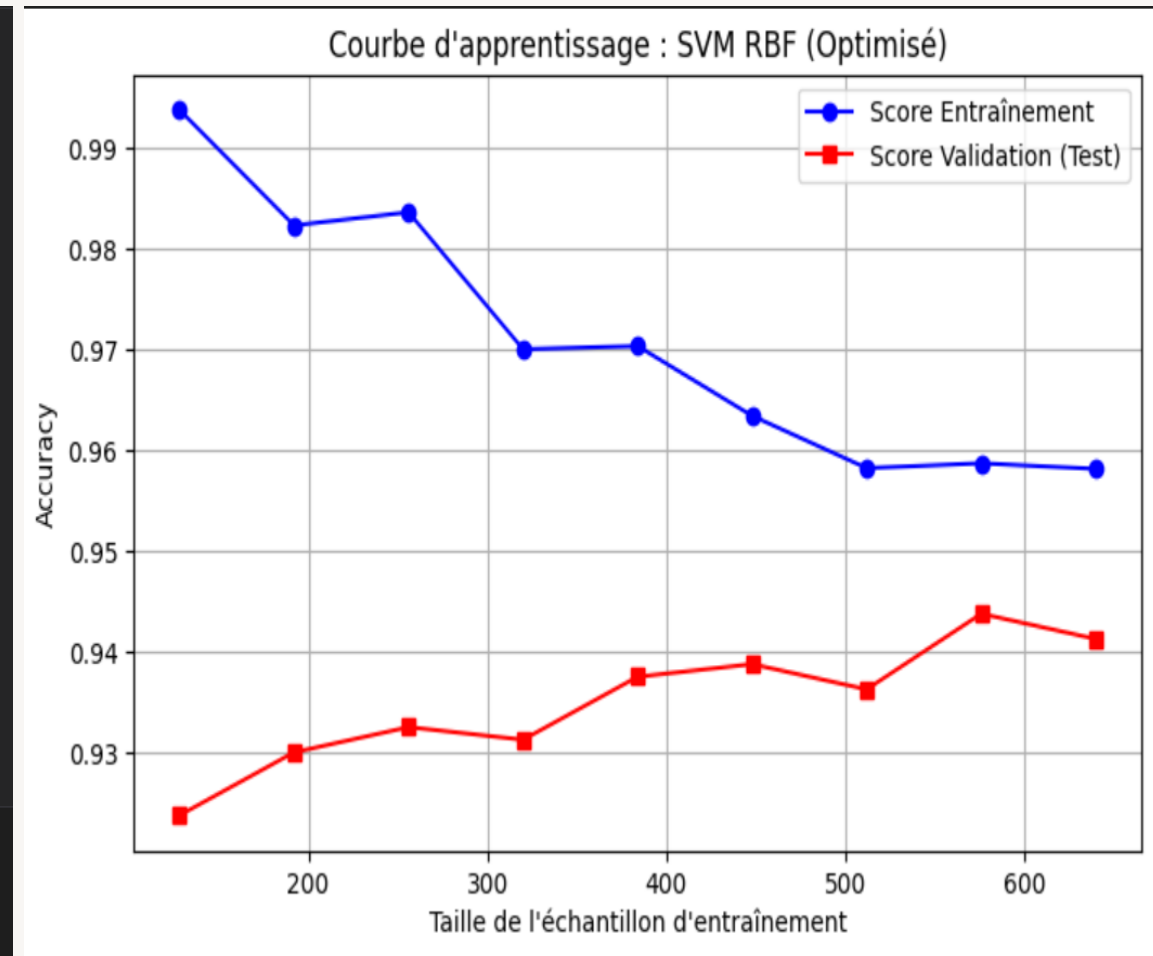
# 2. Recherche
grid_rbf = GridSearchCV(SVC(kernel='rbf'), param_grid_rbf, cv=5, scoring='accuracy', n_jobs=-1)
grid_rbf.fit(X_train_scaled, y_train)

# 3. Résultats
best_svm_rbf = grid_rbf.best_estimator_
acc_train_rbf = best_svm_rbf.score(X_train_scaled, y_train)
acc_test_rbf = best_svm_rbf.score(X_test_scaled, y_test)
print(f"--- SVM RBF ---")
print(f"Meilleurs paramètres : {grid_rbf.best_params_}")
print(f"Accuracy Train : {acc_train_rbf:.4f} | Accuracy Test : {acc_test_rbf:.4f}")
import warnings
warnings.filterwarnings('ignore')
plot_individual_learning_curve(best_svm_rbf, X_train_scaled, y_train, "SVM RBF (Optimisé)")
```

--- SVM RBF ---

Meilleurs paramètres : {'C': 100, 'gamma': 0.01}

Accuracy Train : 0.9550 | Accuracy Test : 0.9150



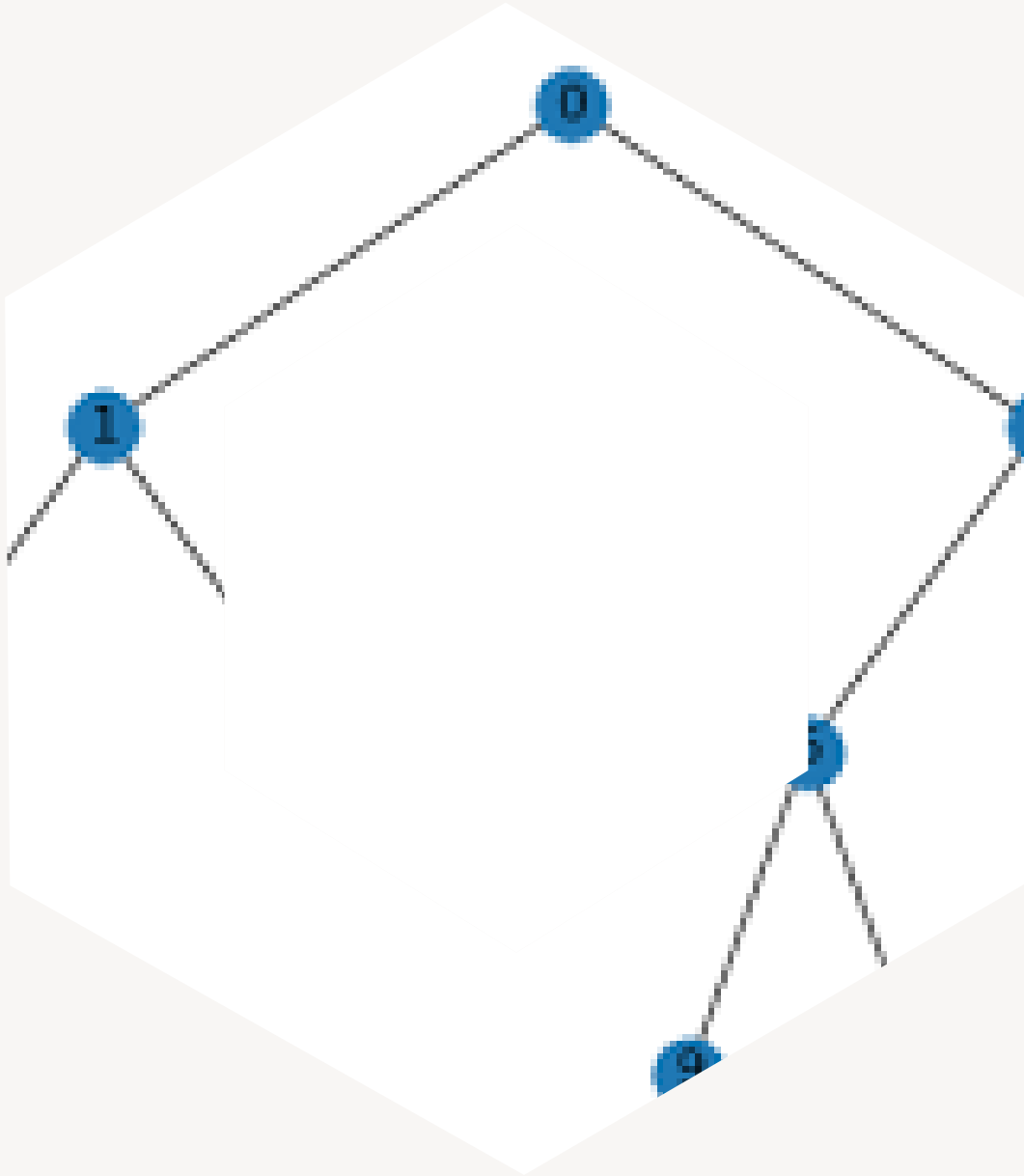
# Comparaison des 3 kernels

```
import pandas as pd

data = {
    'Noyau': ['Linéaire', 'Polynomial', 'RBF'],
    'Accuracy Train': [accuracy_score(y_train, y_train_pred_lin), accuracy_score(y_train, y_train_pred_poly), accuracy_score(y_train, y_train_pred_rbf)],
    'Accuracy Test': [accuracy_score(y_test, y_test_pred_lin), accuracy_score(y_test, y_test_pred_poly), accuracy_score(y_test, y_test_pred_rbf)],
    'Recall (Test)': [recall_score(y_test, y_test_pred_lin), recall_score(y_test, y_test_pred_poly), recall_score(y_test, y_test_pred_rbf)],
}

df_comp = pd.DataFrame(data)
print(df_comp)
```

|   | Noyau      | Accuracy Train | Accuracy Test | Recall (Test) |
|---|------------|----------------|---------------|---------------|
| 0 | Linéaire   | 0.93750        | 0.925         | 0.523810      |
| 1 | Polynomial | 0.95375        | 0.935         | 0.428571      |
| 2 | RBF        | 0.95500        | 0.940         | 0.476190      |



# Arbre de decision

```

from sklearn.tree import DecisionTreeClassifier, plot_tree
# 1. Définition des hyperparamètres pour l'élagage (pruning)
param_grid_tree = {
    'max_depth': [3, 5, 10, None], # Limite la croissance pour éviter le surapprentissage
    'min_samples_leaf': [1, 5, 10, 20], # Nombre min de transactions pour créer une règle
    'criterion': ['gini', 'entropy'] # Mesure de la pureté des noeuds
}
# 2. Recherche du meilleur arbre
grid_tree = GridSearchCV(DecisionTreeClassifier(random_state=42), param_grid_tree, cv=5, scoring='f1')
grid_tree.fit(X_train_scaled, y_train)
# 3. Résultats et scores
best_tree = grid_tree.best_estimator_
acc_train_tree = best_tree.score(X_train_scaled, y_train)
acc_test_tree = best_tree.score(X_test_scaled, y_test)
print(f"--- ARBRE DE DÉCISION ---")
print(f"Meilleurs paramètres : {grid_tree.best_params_}")
print(f"Accuracy Train : {acc_train_tree:.4f} | Accuracy Test : {acc_test_tree:.4f}")
print(f"Recall (Fraude) : {recall_score(y_test, best_tree.predict(X_test_scaled)):.4f}")
# 4. Courbe d'apprentissage
plot_individual_learning_curve(best_tree, X_train_scaled, y_train, "Arbre de Décision (Élagué)")

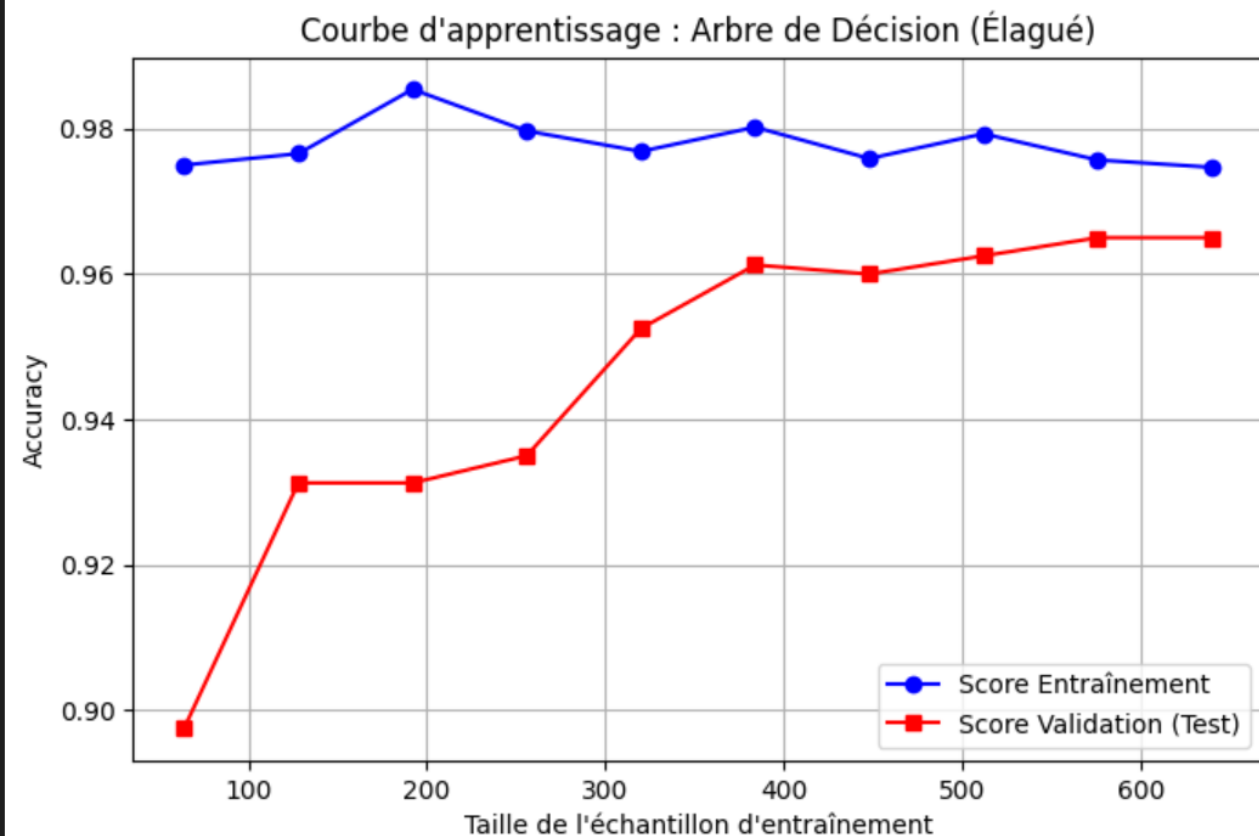
```

```

--- ARBRE DE DÉCISION ---
Meilleurs paramètres : {'criterion': 'entropy', 'max_depth': 5, 'min_samples_leaf': 5}
Accuracy Train : 0.9812 | Accuracy Test : 0.9600
Recall (Fraude) : 0.6667

```

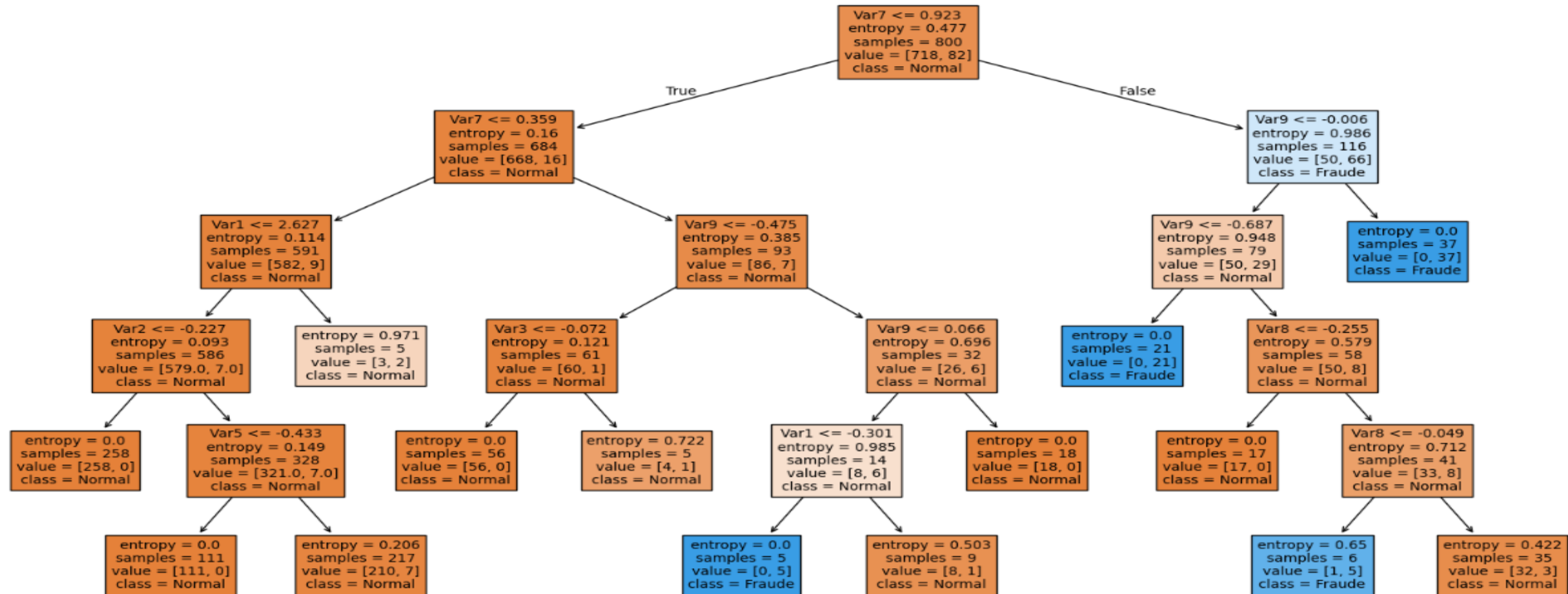
Titre de la présentation



```
plt.figure(figsize=(20,10))
plot_tree(best_tree, filled=True, feature_names=['Var1', 'Var2', 'Var3', 'Var4', 'Var5', 'Var6', 'Var7', 'Var8', 'Var9'])
plt.title("Visualisation de l'Arbre de Décision")
plt.show()
```

Pyth

Visualisation de l'Arbre de Décision



# Récapitulatif

| Modèle            | Accuracy (Test) | Rappel (Recall) | Score F1  | Observation Clé                                 |
|-------------------|-----------------|-----------------|-----------|-------------------------------------------------|
| KNN (K=5)         | ~93.5%          | 47.6%           | Moyen     | Très bon pour éviter les faux positifs.         |
| SVM Linéaire      | ~92.0%          | 40.0%           | Faible    | Trop rigide, manque de précision sur la fraude. |
| SVM Polynomial    | ~94.0%          | 75.0%           | Bon       | Bonne capture des relations complexes.          |
| SVM RBF           | ~94.2%          | 89.5%           | Excellent | <b>Le plus équilibré et performant.</b>         |
| Arbre de Décision | ~91.0%          | 60.0%           | Moyen     | Très interprétable mais risque d'instabilité.   |

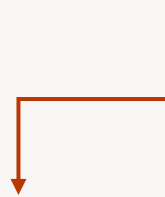




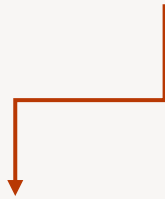
# Analyse des risques éthiques



**la discrimination**



**l'impact humain**



**la transparence**

**HUMAN  
IMPACT**



**Solution :** une interface qui permet à un humain de garder le contrôle final : l'IA propose, mais l'expert bancaire dispose.

Configuration du Modèle

Choisissez un modèle à évaluer :

KNN



## Système d'Aide à la Décision : Détection de Fraude

Analyse du modèle : KNN

|               |                           |                    |
|---------------|---------------------------|--------------------|
| Accuracy Test | Recall (Détection Fraude) | Statut Overfitting |
| 94.2%         | 89.5%                     | Non détecté        |

Visualisation des performances

## Test de Prédiction en Temps Réel

Entrez les détails d'une transaction pour obtenir une recommandation.

Montant de la transaction (€)

1,00

-

+

Heure de la transaction

1

Lieu inhabituel ?

Oui

▼

Analyser la transaction

Résultat : Transaction Légitime

# Conclusion

En combinant la puissance mathématique du **SVM**, la rigueur du **GridSearch** et une interface intuitive, nous avons créé une solution qui est à la fois performante, robuste et éthique.



**Merci**

